

Mycellius — Séance 1 (4h) : Sprint 0

Socle + workflow dev/test/prod + CI « vide mais vert » — guide pas à pas (avec commandes)

Objectif de la séance (ce qu'on veut réussir)

À la fin, vous devez avoir :

- un repo GitHub structuré + branches dev/test/prod
- 3 VM Proxmox (dev/test/prod) prêtes (SSH + sécurité minimale)
- Docker + nginx installés
- une preuve de déploiement : une page statique différente selon l'environnement (DEV/TEST/PROD) servie sur chaque VM
- une CI GitHub Actions minimale qui passe (même si l'application n'existe pas encore)
- une baseline sécurité (CIA + mini registre des risques)

Découpage simple de la séance

- Cadrage MVP + user stories (45–60 min)
- Repo GitHub + branches + règles (30–40 min)
- Proxmox : création VM dev/test/prod (60–80 min)
- SSH + durcissement minimal + Docker + nginx (50–70 min)
- Déploiement “preuve” par branche (30–40 min)
- CI minimale + baseline cybersécurité (30–40 min)

1) Cadrage produit (MVP + 5–8 user stories)

1.1 Définir le MVP (à écrire dans docs/01_cadrage.md)

Écrire :

- Nom : Mycellius
- Objectif (1 phrase) : “Wiki interne pour docs techniques (dev + sysadmin)”
- Rôles : STAGIAIRE, DEV, ADMIN

- IN (MVP) : login, lire page, lister pages, créer/éditer (DEV), tags, recherche simple
- OUT : commentaires, notifications, SSO, pièce jointe, etc.

Rappel — MVP

Le MVP (Minimum Viable Product) est la version la plus petite possible qui apporte déjà une valeur réelle. Ça sert à éviter l'effet "on veut tout faire" et donc à livrer vite une base testable. Dans un TP long, le MVP est vital : il fixe une ligne d'arrivée réaliste et permet de faire des sprints où chaque séance apporte un résultat visible.

1.2 Écrire 5 à 8 user stories (toujours dans docs/01_cadrage.md)

Format :

- US-01 — En tant que ... je veux ... afin de ...
- Critères d'acceptation (Given/When/Then)

Exemples :

- US-01 Auth : se connecter et obtenir un token
- US-02 Wiki : créer une page (DEV)
- US-03 Wiki : lire une page (STAGIAIRE)
- US-04 Wiki : lister pages avec pagination
- US-05 Wiki : rechercher par titre
- US-06 Tags : taguer une page
- US-07 RBAC : stagiaire lecture seule, dev édition, admin gestion

Rappel — User story & critères d'acceptation

Une user story décrit un besoin utilisateur, pas une tâche technique. Les critères d'acceptation transforment ce besoin en "contrat de validation" : on saura objectivement si c'est terminé. C'est très utile en BTS : ça évite les débats flous ("ça marche à peu près") et ça rend l'évaluation plus juste.

1.3 Definition of Done (DoD) minimal (fin de docs/01_cadrage.md)

DoD séance 1 (exemple) :

- docs à jour
- CI verte
- déploiement test OK
- aucun secret dans Git

Rappel — Definition of Done (DoD)

La DoD définit ce que “terminé” veut dire. Sans DoD, on se ment facilement (ex : “ça compile chez moi”). Avec une DoD, on impose une discipline proche entreprise : tests, doc, review, déploiement, preuves.

2) GitHub : repo, structure, branches, règles

2.1 Créer le repo et l’arborescence (avec commandes)

Vous pouvez le faire de deux façons : (A) via l’interface GitHub + commandes Git, ou (B) tout en ligne de commande avec GitHub CLI (gh).

A) Méthode simple (recommandée en BTS) : GitHub (web) + Git (local)

1) **Sur GitHub** : créez un nouveau dépôt (Repository) nommé par exemple : mycellius-<prenom>.

2) **Sur votre poste** : ouvrez un terminal dans un dossier de travail (ex : Documents/bts) puis exécutez :

```
mkdir mycellius
cd mycellius
git init
```

Rappel — git init

git init transforme un dossier en dépôt Git local. À partir de là, Git peut suivre l’historique des fichiers (versions) et vous permettre de revenir en arrière ou de comparer les changements.

3) **Créez l’arborescence du projet** :

```
mkdir -p api web mobile infra docs ci .github/workflows infra/static
```

Rappel — Arborescence de monorepo

Ici on utilise un seul dépôt qui contient l’API, le web, le mobile, l’infrastructure et la documentation. C’est pratique pour un TP transversal : une seule URL, une seule CI, et une vision complète du projet.

4) **Créez les fichiers de base** :

```
touch README.md .gitignore
touch docs/01_cadrage.md docs/02_architecture.md docs/
03_securite_baseline.md
touch infra/static/index.html
touch ci/validate_repo.sh
touch .github/workflows/ci.yml
```

5) Ajoutez un contenu minimal :

README.md (exemple de 5 lignes) :

```
# Mycellius
Wiki interne (Web + Mobile) pour documents techniques.

## Structure
api/ web/ mobile/ infra/ docs/

.gitignore (minimum) :

.env
node_modules/
.DS_Store
.idea/
```

Rappel — .gitignore

Le fichier .gitignore dit à Git quels fichiers NE DOIVENT PAS être versionnés. On y met typiquement : secrets (.env), dépendances (node_modules), dossiers IDE (.idea). C'est une règle d'hygiène essentielle : un secret commité est souvent un incident de sécurité.

6) Premier commit :

```
git add .
git commit -m "chore: init repo structure"
```

Rappel — commit

Un commit est une “photo” de l'état du projet à un instant T, avec un message. C'est l'unité de base de l'historique : on peut comparer, revenir en arrière, et relire l'évolution.

7) Lier le dépôt local à GitHub et pousser (push).

Sur GitHub, copiez l'URL du dépôt (HTTPS) puis :

```
git branch -M dev
git remote add origin <URL_DU_REPO>
git push -u origin dev
```

Rappel — remote / push

Un remote est un dépôt distant (ici GitHub). push envoie vos commits locaux vers le dépôt distant. Le -u enregistre la branche distante comme destination par défaut (plus simple ensuite).

B) Option : GitHub CLI (gh) (si disponible)

Si l'outil gh est installé et authentifié :

```
gh repo create mycellius-<prenom> --private --source=. --remote=origin --push
```

Rappel — GitHub CLI (gh)

gh est un outil en ligne de commande pour GitHub. Il permet de créer un repo, ouvrir des PR, gérer des issues... sans passer par le navigateur. C'est pratique, mais optionnel en BTS.

2.2 Créer les branches dev, test, prod (avec commandes)

Dans ce TP, la règle est la suivante :

- On travaille au quotidien sur dev.
- On intègre/valide sur test via Pull Request (PR).
- On publie une version stable sur prod via Pull Request (PR).

Rappel — PR (Pull Request)

Une Pull Request est une demande de fusion : vous proposez d'intégrer du code d'une branche vers une autre. C'est le moment où on relit, on vérifie la CI, et on valide avant d'intégrer. Dans le monde pro, c'est l'étape clé pour éviter d'introduire des bugs dans les branches stables.

Commandes pour créer test et prod depuis dev (après votre premier push) :

```
git checkout dev
git pull
git checkout -b test
git push -u origin test
git checkout dev
git checkout -b prod
git push -u origin prod
git checkout dev
```

Rappel — branches

Une branche est une ligne de développement séparée. dev sert à construire ; test sert à valider ; prod sert à livrer stable. Chaque environnement Proxmox suivra sa branche (git pull sur la VM).

Option : vérifier que les branches existent bien (local + distant) :

```
git branch
git branch -r
```

En complément, sur GitHub (Settings → Branches), activez :

- PR obligatoire vers test et prod (branch protection rules).
- 1 review minimum (si possible).

- Require status checks (CI) lorsque le workflow est en place.

Rappel — branch protection rules

Les protections de branche empêchent les erreurs humaines (push direct sur test/prod). Elles forcent l'usage des PR, qui jouent le rôle de "porte de sécurité" avant intégration.

3) Proxmox : créer 3 VM (DEV / TEST / PROD)

3.1 Combien de VM ?

➡ 3 VM recommandées : mycellius-dev, mycellius-test, mycellius-prod

Rappel — Proxmox & VM

Proxmox est une plateforme de virtualisation : elle héberge des VM (machines virtuelles). Une VM est un "serveur complet" isolé (OS + CPU/RAM/disque). C'est idéal en pédagogie : chaque environnement est séparé, donc on peut casser DEV sans toucher PROD.

3.2 Configuration recommandée (Ubuntu Server LTS)

Pour chaque VM :

- 2 vCPU
- 4 Go RAM (2 Go possible si machine limitée, mais moins confortable)
- 20–30 Go disque
- réseau : IP fixe si possible (sinon DHCP réservé)

Nommer :

- mycellius-dev
- mycellius-test
- mycellius-prod

Résultat attendu : vous avez 3 VM démarrées, joignables sur le réseau.

4) Accès SSH + durcissement minimal + Docker + nginx

4.1 SSH par clé (sur chaque VM)

Sur le poste (si pas déjà fait) :

```
ssh-keygen -t ed25519 -C "bts-sio-mycellius"
```

Copier la clé sur chaque VM :

```
ssh-copy-id user@IP_VM
```

Connexion :

```
ssh user@IP_VM
```

Rappel — SSH & clé publique/privée

SSH permet d'administrer un serveur à distance. La méthode la plus sûre est l'auth par clé : la clé privée reste sur le PC, la clé publique va sur le serveur. Résultat : on évite les mots de passe faibles et les attaques par bruteforce.

4.2 Durcissement minimal (sur chaque VM)

```
sudo apt update && sudo apt -y upgrade
sudo apt -y install ufw
sudo ufw allow OpenSSH
sudo ufw enable
```

Option (si vous maîtrisez) : désactiver login par mot de passe SSH.

Rappel — UFW / durcissement minimal

Un serveur exposé sans règles de firewall est un serveur "ouvert". UFW est un pare-feu simple qui permet de limiter les ports. On commence petit : autoriser SSH, puis plus tard HTTP/HTTPS. Le durcissement minimal dès le départ installe un réflexe Bloc 3 : "sécurité by design".

4.3 Installer Docker + Compose (sur chaque VM)

```
sudo apt -y install docker.io docker-compose-plugin
sudo usermod -aG docker $USER
newgrp docker
docker --version
docker compose version
```

Rappel — Docker & Docker Compose

Docker lance des services dans des conteneurs : c'est reproductible. Docker Compose décrit un ensemble de services dans un fichier YAML (ex : MySQL + API). L'intérêt pédagogique : tout le monde lance les mêmes services avec 1 commande, donc moins de "ça marche pas chez moi".

4.4 Installer nginx (ou le laisser si vous préférez python http.server)

```
sudo apt -y install nginx
sudo systemctl enable --now nginx
```

Test : navigateur → `http://IP_VM`

Rappel — nginx

nginx est un serveur web très courant. Ici il sert surtout à prouver le déploiement par environnement : on va publier un fichier HTML différent sur DEV/TEST/PROD, et vérifier visuellement que la branche correspond bien à la VM.

5) Déploiement “preuve” : une page statique différente sur chaque VM selon la branche

5.1 Créer le dossier `infra/static/` dans le repo

Sur ta branche dev, ajoute :

- `infra/static/index.html` (avec badge “DEV” par exemple)
- `infra/static/style.css` (optionnel)

Puis tu feras une version “TEST” et “PROD” via merge/promotion (ou fichier config).

Exemple `infra/static/index.html` (DEV) :

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Mycellius — DEV</title>
</head>
<body style="font-family: system-ui; padding: 24px;">
  <h1>Mycellius</h1>
  <p><b>ENV :</b> DEV</p>
  <p>Preuve : cette VM suit la branche <code>dev</code>.</p>
</body>
</html>
```

Rappel — “preuve de déploiement”

Avant même l’API, on prouve que le pipeline humain fonctionne : push → PR → merge → git pull sur la VM. Si ça marche avec une page HTML, alors ce sera la même mécanique plus tard avec l’API et les front.

5.2 Cloner la branche sur chaque VM (une fois)

Sur la VM DEV :

```
sudo rm -rf /var/www/html
sudo git clone -b dev <URL_DU_REPO> /var/www/html
sudo systemctl reload nginx
```

Sur la VM TEST :

```
sudo rm -rf /var/www/html
sudo git clone -b test <URL_DU_REPO> /var/www/html
sudo systemctl reload nginx
```

Sur la VM PROD :

```
sudo rm -rf /var/www/html
sudo git clone -b prod <URL_DU_REPO> /var/www/html
sudo systemctl reload nginx
```

Ensuite, à chaque mise à jour :

```
cd /var/www/html
sudo git pull
sudo systemctl reload nginx
```

Résultat attendu :

- `http://IP_DEV` affiche “DEV”
- `http://IP_TEST` affiche “TEST”
- `http://IP_PROD` affiche “PROD”

(Pour TEST/PROD, tu obtiens la différence en promouvant dev→test puis test→prod et en adaptant le titre/badge sur chaque branche.)

6) CI GitHub Actions minimale (pipeline “vide mais vert”)

6.1 Ajouter un script de validation simple (sans dépendances)

ATTENTION !!!!

Windows : faites ces commandes dans WSL ou Git Bash.

Si votre CI échoue avec des erreurs de script, convertissez le fichier en LF (pas CRLF) et recommittez.

Créer ci/validate_repo.sh :

```
#!/usr/bin/env bash
set -euo pipefail

test -f README.md
test -d docs
test -d infra
test -d .github/workflows

test -f docs/01_cadrage.md
test -f docs/02_architecture.md || true
test -f docs/03_securite_baseline.md || true

echo "OK: structure de repo minimale présente."
```

Puis rendre exécutable :

```
chmod +x ci/validate_repo.sh
```

6.2 Ajouter le workflow .github/workflows/ci.yml

```
name: CI

on:
  push:
    branches: [dev, test, prod]
  pull_request:
    branches: [test, prod]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Validate repository structure
        run: bash ci/validate_repo.sh
```

Rappel — CI (Intégration Continue)

La CI automatise des vérifications à chaque push/PR. Même si l'app n'existe pas encore, une CI minimale empêche la dérive ("plus personne ne met de doc", "on oublie un fichier"). Plus tard, on remplacera ce job par : build API + tests + build web + tests, etc.

7) Baseline cybersécurité (CIA + registre risques)

7.1 Écrire docs/03_securite_baseline.md

Inclure :

- CIA (Confidentialité / Intégrité / Disponibilité)
- menaces principales
- mesures minimales
- mini tableau risques

Rappel — CIA (Confidentialité, Intégrité, Disponibilité)

C'est une grille simple pour réfléchir à la sécurité : Confidentialité = empêcher l'accès non autorisé (auth, rôles, secrets). Intégrité = empêcher la modification non autorisée (validation, logs, migrations, CI). Disponibilité = assurer que le service reste accessible (backup, relance, monitoring minimal). C'est parfait pour le Bloc 3 car c'est structurant et facilement "prouvable".

Exemple mini registre risques (dans le doc) :

Risque	Impact	Vraisemblance	Mesures	Preuve
Secrets committés	Fort	Moyen	.env ignoré, variables env	.gitignore, CI
Compte compromis	Fort	Moyen	SSH par clé, RBAC plus tard	config ssh, policy
Perte DB	Fort	Moyen	backup/restore	procédure + test

Rappel — Registre des risques

Un registre n'est pas un "papier pour faire joli" : c'est une liste actionnable. Chaque risque doit avoir une mesure ET une preuve. En entreprise, c'est ce qui vous protège quand on demande : "qu'avez-vous prévu si... ?".

Check final (fin de séance 1)

- ✓ Repo GitHub : structure + README + docs
- ✓ Branches : dev/test/prod + protections minimum (PR vers test/prod)
- ✓ 3 VM : SSH OK + UFW OK + Docker OK + nginx OK
- ✓ Déploiement preuve : HTML différent sur DEV/TEST/PROD
- ✓ CI : workflow GitHub Actions vert
- ✓ Sécurité : CIA + registre risques (mini)

Étape 1 (Séance 1) — Git + GitHub : installer/configurer + repo + branches

Où ? sur ton poste Wi-Fi

1A) Vérifier Git

```
git --version
```

✓ attendu : une version s'affiche

1B) Configurer Git (une seule fois)

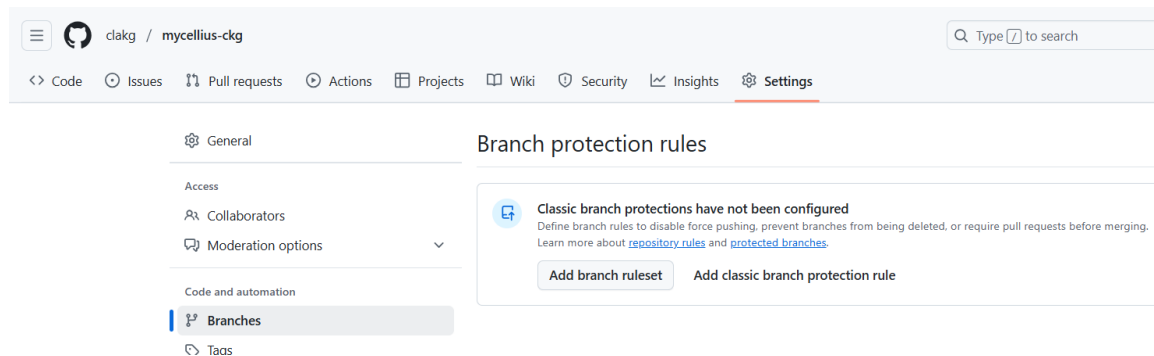
```
git config --global user.name "Prenom Nom"
```

```
git config --global user.email "email@domaine.tld"
```

```
git config --global init.defaultBranch dev
```

1C) Créer le repo sur GitHub (navigateur)

New repository → mycellius-<classe>-<prenom>



Ne pas générer README (on le fait en local)

1D) Créer la structure en local + push sur dev

```
mkdir mycellius
```

```
cd mycellius
```

```
git init
```

```
mkdir -p api web mobile infra infra/static docs ci .github/workflows
```

```
touch README.md .gitignore
```

```
touch docs/01_cadrage.md docs/02_architecture.md docs/03_securite_baseline.md
```

```
touch infra/static/index.html
```

```
touch ci/validate_repo.sh
```

```
touch .github/workflows/ci.yml
```

```
git add .
```

```
git commit -m "chore: init repo structure"
```

1E) Créer test et prod

1) Lier le dépôt local à GitHub et pousser dev

```
git branch -M dev
git remote add origin <URL_DU_REPO_GITHUB>
git push -u origin dev
```

2) Créer test et prod depuis dev (après le premier push)

```
git checkout dev
git pull
git checkout -b test
git push -u origin test

git checkout dev
git checkout -b prod
git push -u origin prod

git checkout dev
```

1F — Protections de branches GitHub (PR obligatoire)

Settings → Branches → protection rule sur test et prod :

Pré-requis

Les branches test et prod existent déjà sur le dépôt (tu les as “push” au moins une fois).

A) Ajouter une règle pour test

- Va sur ton dépôt GitHub.
- Clique sur l’onglet Settings (paramètres du repo).
- Dans le menu de gauche, va dans Branches.
- Dans la section Branch protection rules, clique Add rule.
- Dans Branch name pattern, tape :

> test

(ou test* si tu veux couvrir des variantes, mais ici test suffit)

```
clakg@CKG-FORMATION MINGW64 ~/mycellius (test)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 261 bytes | 261.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: Bypassed rule violations for refs/heads/test:
remote:
remote: - Changes must be made through a pull request.
remote:
To https://github.com/clakg/mycellius-ckg.git
9471f35..9f654ba test -> test
```

- Coche :

- ☒ Require a pull request before merging
(Recommandé) Dans la section qui apparaît, coche aussi :
- ☒ Require approvals → mets **1** (1 validation avant merge)
Clique sur Create ou Save changes (bouton en bas).

```
clakg@CKG-FORMATION MINGW64 ~/mycellius (test)
• $ git checkout dev
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.

clakg@CKG-FORMATION MINGW64 ~/mycellius (dev)
• $ echo "test on DEV" >> README.md

clakg@CKG-FORMATION MINGW64 ~/mycellius (dev)
• $ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

clakg@CKG-FORMATION MINGW64 ~/mycellius (dev)
• $ git commit -m "docs: test protection"
[dev 8cb914d] docs: test protection
1 file changed, 1 insertion(+)

clakg@CKG-FORMATION MINGW64 ~/mycellius (dev)
• $ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 265 bytes | 265.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/clakg/mycellius-ckg.git
9471f35..8cb914d dev -> dev
```

Rappel — PR obligatoire

Ça bloque le merge “direct” sur test. Toute modification doit passer par une Pull Request (ex : dev → test).

B) Ajouter une règle pour prod

- Toujours dans Settings → Branches.
- Add rule à nouveau.
- Dans Branch name pattern, tape :
 > prod
- Coche :
 - ☒ Require a pull request before merging
 - ☒ Require approvals → 1 (ou 2 si tu es en mode “entreprise”, mais 1 suffit en BTS)
- Clique Save changes.

C) Vérifier que ça marche (test rapide)

- Test 1 : vérification visuelle

- Va dans Code → branches

- Les branches test et prod doivent afficher un petit indicateur de protection (souvent une icône / texte "Protected").

Test 2 : essayer de pousser sur test (optionnel)

En local :

git checkout test

echo "test" >> README.md

git add README.md

git commit -m "docs: test protection"

git push

 Résultat attendu : GitHub doit refuser si tu as aussi coché une règle qui empêche directement les push (selon réglages).

 Attention : "PR obligatoire" n'empêche pas forcément le push. Ça empêche surtout de merger sans PR sur GitHub.

Ce qu'on fait maintenant (étape suivante, très concrète)

1) On arrête de travailler sur test

- Tu te remets sur dev :

git checkout dev

2) On fait la bonne manipulation : PR dev → test

- Tu pushes tes changements sur dev

- Puis sur GitHub : Pull requests → New pull request → base: test ← compare: dev

- Tu crées la PR, et tu merges vers test (si autorisé)

- Conflit entre branch test et dev...

Méthode recommandée : résoudre en local (sur ton poste, Git Bash)

Où ? Poste Wi-Fi (ton PC) — terminal Git Bash ou terminal VS Code en Git Bash.

Reviens sur dev et mets à jour :

git checkout dev

git pull origin dev

git fetch origin

- Fusionne test dans dev (c'est ça qui va déclencher le conflit localement) :

```
git merge origin/test
```

- Git va te dire qu'il y a conflit sur README.md.

- Ouvre README.md dans VS Code : tu verras des marqueurs comme :

```
<<<<<<< HEAD
```

```
...
```

```
=====
```

```
...
```

```
>>>>>>> origin/test
```

Dans VS Code tu as généralement des boutons :

Accept Current Change (version dev)

Accept Incoming Change (version test)

Accept Both

Compare Changes

👉 Pour un README, le plus simple : Accept Both puis tu réorganises proprement le texte.

Une fois le fichier propre, valide la résolution :

```
git add README.md
```

```
git commit -m "chore: resolve conflict dev vs test"
```

```
git push origin dev
```

✅ Résultat attendu : la PR n'a plus de conflit, et le bouton "Merge" redevient disponible (sous réserve des règles de review).

Petit contrôle à faire pour être 100% sûre (1 minute)

Dans Git Bash, colle-moi la sortie de :

```
git branch
```

Je veux juste vérifier que tu es bien revenu sur dev avant de continuer.

=> OK

Rappel — pourquoi dev/test/prod dès maintenant ?

Parce qu'on va faire correspondre :

VM DEV ↔ branche dev

VM TEST ↔ branche test

VM PROD ↔ branche prod

Et la “promotion” se fera par PR (dev→test→prod). C’est exactement le workflow que tu veux enseigner.

➡ Checkpoint pour passer à l’étape suivante (Proxmox/VM) :

Qu'as tu à la sortie de :

git branch -r

et dis-moi si la règle “PR obligatoire” est bien activée sur test et prod.

Étape 2 — Proxmox : créer 3 VM Ubuntu (mycellius-dev / test / prod)

2A) Où faire les manip ?

Sur ton poste Wi-Fi (navigateur) : interface web Proxmox https://IP_PROXMOX:8006

Dans Proxmox : création VM

(Pas besoin du poste client)

2B) Paramètres recommandés (identiques pour les 3 VM)

Pour chaque VM (Ubuntu Server LTS) :

CPU : 2 vCPU

RAM : 4 Go (2 Go si tu es serrée)

Disque : 25 Go

Réseau : bridge habituel (souvent vmbr0)

DNS/Gateway : par défaut si DHCP, sinon en statique

Nom :

mycellius-dev

mycellius-test

mycellius-prod

Rappel — pourquoi 3 VM ?

Parce qu'on veut une correspondance simple et pédagogique : 1 VM = 1 environnement = 1 branche.

2C) Création de la VM 1 : mycellius-dev

Dans Proxmox :

- Clique sur ton node (serveur)
- Create VM
- General
- VM ID : (laisse proposé)
- Name : mycellius-dev
- OS
- Sélectionne l'ISO Ubuntu Server LTS (22.04 ou 24.04)
- System
- Defaults OK (OVMF/BIOS selon ton infra, ne change rien si tu ne sais pas)
- Disks
- 25 GB (ou 20–30)
- CPU
- 2 cores
- Memory
- 4096 MB
- Network
- Bridge : vmbr0 (le bridge principal)
- Finish

 Démarre la VM (Start) puis installe Ubuntu (setup classique).

Pendant l'installation Ubuntu (important)

- Crée un utilisateur (ex : bts ou ton prénom)
- Active OpenSSH server quand Ubuntu propose la case à cocher 
- Rappel — OpenSSH server

Si tu coches ça, tu pourras administrer la VM depuis ton poste Wi-Fi en SSH, sans passer par la console Proxmox.

2D) Répéter pour mycellius-test et mycellius-prod

Même paramètres, juste le nom.

2E) Vérification (obligatoire avant l'étape suivante SSH)

Sur la console Proxmox de chaque VM, récupère l'IP :

ip a

Tu dois obtenir 3 IP (une par VM).

Checkpoint à me donner (pour continuer)

Copie/colle ici :

IP de mycellius-dev :

IP de mycellius-test :

IP de mycellius-prod :

et confirme que tu as bien coché OpenSSH server à l'installation (oui/non)

La méthode propre en TP (Proxmox)

1) Créer et préparer la VM "base" (mycellius-dev)

Où ? Proxmox + console VM

Tu installes Ubuntu sur mycellius-dev avec :

- OpenSSH Server 
- user (ex: bts)
- réseau (DHCP ou statique)
- mise à jour (apt update/upgrade)
- (optionnel mais conseillé) installer déjà git, ufw, docker, nginx

👉 Une fois que mycellius-dev fonctionne, tu passes aux clones.

2) Stopper la VM avant clonage

Où ? Proxmox

- mycellius-dev → Shutdown (pas "reboot", pas "pause")
- Attendre qu'elle soit stopped

Rappel — pourquoi arrêter avant cloner ?

Pour éviter de cloner un système en plein changement (risque de corruption, identifiants réseau dupliqués, etc.).

3) Cloner vers mycellius-test et mycellius-prod

Où ? Proxmox (clic droit sur la VM)

- Clic droit sur mycellius-dev → Clone
- Name : mycellius-test
- Puis recommencer → Name : mycellius-prod
- Linked clone ou Full clone ?
- Full clone  (recommandé TP) : indépendant, plus robuste
- Linked clone : économise disque mais dépend du parent (moins simple)

4) Après clonage : changer nom + IP (obligatoire)

Si tu clones, tu dois changer au minimum :

- hostname (nom machine)
- IP (si statique) ou réserver une IP via DHCP
- (optionnel) régénérer l'ID machine (machine-id) pour éviter des collisions

4A) Changer le hostname

Où ? Dans chaque VM clonée (console / SSH)

- Sur mycellius-test :

```
sudo hostnamectl set-hostname mycellius-test
```

- Sur mycellius-prod :

```
sudo hostnamectl set-hostname mycellius-prod
```

- Vérifie :

```
hostname
```

Dès que c'est OK, on passe à Étape 3 : SSH depuis ton poste Wi-Fi + durcissement (UFW) + Docker + nginx

Étape 3 — SSH (poste Wi-Fi → VM) + UFW + Docker + nginx (sur mycellius-dev)

3A) Récupérer l'IP de la VM (dans la console Proxmox de la VM)

Où ? Dans la VM (console Proxmox)

hostname -I

➡ Note l'IP (ex : 192.168.1.50).

Résultat attendu : tu as une IP exploitable.

3B) Depuis ton poste Wi-Fi : vérifier que tu peux joindre la VM

Où ? Poste Wi-Fi (Windows / PowerShell)

ping <IP_DE_LA_VM>

- Résultat attendu :

réponses = parfait

pas de réponse = pas grave parfois (ICMP bloqué), on testera SSH ensuite

3C) Créer une clé SSH sur ton poste (si tu n'en as pas)

Où ? Poste Wi-Fi (VS Code terminal / PowerShell)

- Vérifie si tu as déjà une clé :

```
dir $env:USERPROFILE\.ssh
```

- Si tu vois id_ed25519 et id_ed25519.pub, passe à l'étape suivante.

- Sinon, crée la clé :

```
ssh-keygen -t ed25519 -C "bts-sio-mycellius"
```

- Appuie sur Entrée pour le chemin par défaut

- Mets une passphrase si tu veux (optionnel en TP)

Rappel — clé SSH

clé privée = reste sur ton PC (ne jamais l'envoyer)

clé publique = va sur la VM (autorise la connexion)

3D) Copier ta clé publique sur la VM (pour SSH sans mot de passe)

```
PS C:\WINDOWS\system32> ssh-keygen -t ed25519 -C "bts-sio-mycellius"  
Generating public/private ed25519 key pair.
```

```

Enter file in which to save the key (C:\Users\clakg/.ssh/id_ed25519):
Created directory 'C:\\Users\\clakg/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\clakg/.ssh/id_ed25519
Your public key has been saved in C:\Users\clakg/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:4wHrxfOIC5IzUUg3PWVlla2h184R3I2/hlyvv5AvKI8 bts-sio-mycellius
The key's randomart image is:
+--[ED25519 256]--+
|      . =. |
|      B + |
|      . oo.* |
|      + .+o=.. |
|      . S oo+... |
|      . + =oo..o. |
|      . * oo+o. |
|      . o.*oo +o |
|      +o.E+o. o+. |
+----[SHA256]-----+

```

```

PS C:\WINDOWS\system32> ssh-keygen -t ed25519 -C "bts-sio-mycellius"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\clakg/.ssh/id_ed25519):
Created directory 'C:\\Users\\clakg/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\clakg/.ssh/id_ed25519
Your public key has been saved in C:\Users\clakg/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:4wHrxfOIC5IzUUg3PWVlla2h184R3I2/hIyvv5AvKI8 bts-sio-mycellius
The key's randomart image is:
+--[ED25519 256]--+
|      . =. |
|      B + |
|      . oo.* |
|      + .+o=.. |
|      . S oo+... |
|      . + =oo..o. |
|      . * oo+o. |
|      . o.*oo +o |
|      +o.E+o. o+. |
+----[SHA256]-----+

```

Tu as 2 façons (choisis celle qui marche chez toi).

Option 1 — ssh-copy-id (si dispo)

```
ssh-copy-id <user>@<IP_DE_LA_VM>
```

Option 2 — Windows (toujours dispo) avec type + SSH

- Afficher la clé publique :

```
type $env:USERPROFILE\.ssh\id_ed25519.pub
```

- Copier la ligne complète affichée, puis :

```
ssh <user>@<IP_DE_LA_VM>
```

```
mkdir -p ~/.ssh
```

```
vi ~/.ssh/authorized_keys
```

- Colle ta clé, enregistre :

- Puis sécurise :

```
chmod 700 ~/.ssh
```

```
chmod 600 ~/.ssh/authorized_keys
```

```
exit
```

- Test final SSH (depuis ton poste)

```
ssh <user>@<IP_DE_LA_VM>
```

✅ Résultat attendu : tu entres dans la VM sans demander le mot de passe (ou seulement la passphrase de ta clé si tu en as mis une).

3E) Durcissement minimal : UFW (dans la VM via SSH)

Où ? Dans la VM (session SSH)

1. Mise à jour :

```
sudo apt update && sudo apt -y upgrade
```

2. Installer UFW + autoriser SSH :

```
sudo apt -y install ufw
```

```
sudo ufw allow OpenSSH
```

```
sudo ufw enable
```

```
sudo ufw status
```

✅ Résultat attendu :

```
UFW "active"
```


règle OpenSSH "ALLOW"

Rappel — UFW

Pare-feu simple : on ouvre seulement ce dont on a besoin. Ici SSH (et bientôt HTTP).

```
clarisse@mycellius-dev:~$ sudo apt -y install ufw
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
ufw est déjà la version la plus récente (0.36.2-6).
ufw passé en « installé manuellement ».
0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
clarisse@mycellius-dev:~$ sudo ufw allow OpenSSH
Rules updated
Rules updated (v6)
clarisse@mycellius-dev:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
clarisse@mycellius-dev:~$ sudo ufw status
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
```

3F) Installer Docker + Compose (dans la VM via SSH)

Installer Docker via le dépôt officiel (avec Compose v2)

Où ? Dans la VM (ta session SSH)

1. Nettoyage (au cas où) :

```
sudo apt remove -y docker.io docker-doc docker-compose docker-compose-v2 podman-docker
containerd runc || true
```

2. Prérequis + ajout clé GPG :

```
sudo apt update
```

```
sudo apt install -y ca-certificates curl gnupg
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/
apt/keyrings/docker.gpg
```

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

3. Ajouter le dépôt Docker :

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \

$(. /etc/os-release && echo ${UBUNTU_CODENAME:-$VERSION_CODENAME}) stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
```

4. Installer Docker + Compose plugin :

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

5. Donner les droits Docker à ton user (sinon tu devras faire sudo à chaque commande docker) :

```
sudo usermod -aG docker $USER

newgrp docker
```

Vérifier :

```
docker --version
docker compose version
docker run --rm hello-world
```



Résultat attendu : docker compose version marche et hello-world s'exécute.
Résultat attendu : message "Hello from Docker!"

```
clarisse@mycellius-dev:~$ docker --version
Docker version 29.1.2, build 890dcca
clarisse@mycellius-dev:~$ docker compose version
Docker Compose version v5.0.0
clarisse@mycellius-dev:~$ docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d200f90: Download complete
Digest: sha256:f7931603f70e13dbd844253370742c4fc4202d290c80442b2e68706d8f33ce26
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

3G) Installer nginx (dans la VM via SSH)

```
sudo apt -y install nginx  
sudo systemctl enable --now nginx
```

Ouvrir le port HTTP dans UFW :

```
sudo ufw allow 'Nginx Full'  
sudo ufw status
```

```
clarisse@mycellius-dev:~$ sudo ufw status  
Status: active
```

To	Action	From
--	-----	----
OpenSSH	ALLOW	Anywhere
Nginx Full	ALLOW	Anywhere
OpenSSH (v6)	ALLOW	Anywhere (v6)
Nginx Full (v6)	ALLOW	Anywhere (v6)

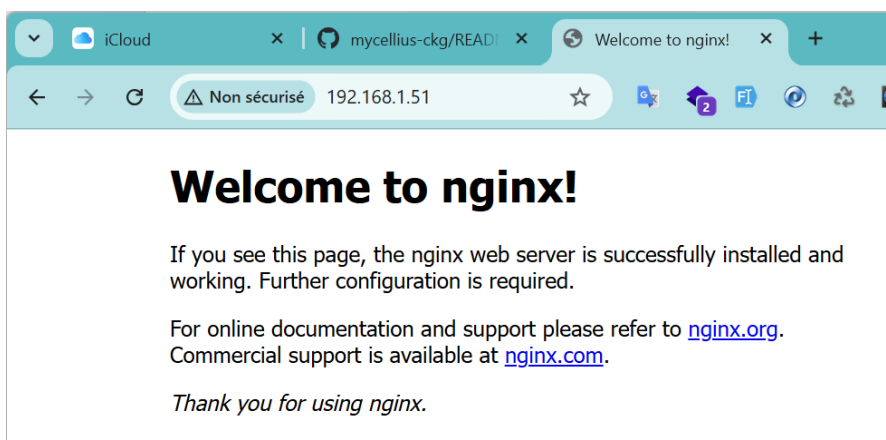
✓ Résultat attendu :

- nginx = active (running)
- UFW autorise Nginx
- Test navigateur (depuis ton poste Wi-Fi)

Va sur :

http://<IP_DE_LA_VM>

✓ Résultat attendu : page nginx par défaut.



On va faire dans cet ordre (propre et rapide) :

- Cloner mycellius-dev vers mycellius-test et mycellius-prod (dans Proxmox)
- Changer hostname + (optionnel) régénérer machine-id sur les clones
- Déployer la preuve DEV/TEST/PROD : chaque VM sert une page différente car elle suit sa branche Git

Étape 4 — Clonage Proxmox : dev → test + prod

4A) Stopper la VM dev avant clonage

Où ? Proxmox (web UI)

Sélectionne mycellius-dev

Clique Shutdown

Attends qu'elle soit Stopped

Résultat attendu : la VM est éteinte.

Rappel — pourquoi éteindre ?

Pour cloner un disque “propre” et éviter de cloner un système en plein changement.

4B) Cloner vers mycellius-test

Où ? Proxmox

Clic droit sur mycellius-dev → Clone

Mode : Full Clone (recommandé TP)

Name : mycellius-test

Laisse le reste par défaut

Valide

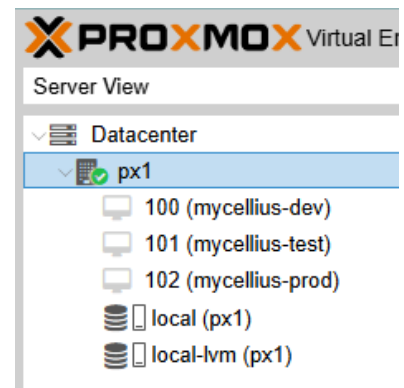
4C) Cloner vers mycellius-prod

Recommence :

Clone → Full clone

Name : mycellius-prod

☒ Résultat attendu : 3 VM existent dans Proxmox.



4D) Démarrer test puis prod

Où ? Proxmox

Start mycellius-test

Start mycellius-prod

Étape 5 — Dans chaque clone : hostname + (recommandé) machine-id

5A) Changer le hostname

Où ? Dans chaque VM (console Proxmox ou SSH)

Dans mycellius-test

```
sudo hostnamectl set-hostname mycellius-test
```

Dans mycellius-prod

```
sudo hostnamectl set-hostname mycellius-prod
```

Vérifie :

```
hostname
```



attendu : le bon nom s'affiche.

5B) (Recommandé) Régénérer l'identité machine

Rappel — machine-id

En clonage, plusieurs VM peuvent avoir le même identifiant interne, ce qui peut provoquer des comportements bizarres dans certains services. Là on évite les ennuis.

Où ? Dans test puis dans prod

```
sudo rm -f /etc/machine-id  
sudo systemd-machine-id-setup  
sudo reboot
```

5C) Récupérer les IP des 3 VM

Où ? Dans chaque VM (console Proxmox)

```
hostname -I
```

Note :

IP dev :

IP test :

IP prod :

✅ attendu : 3 mêmes IP statiques

On va maintenant configurer les IPs statiques des VM mycellius-test et mycellius-prod.

Pour cela :

On va récupérer les informations sur mycellius-dev (IP, GW, NETWORK)

- mycellius-dev :

> IP : 192.168.1.51 (ip a)

> GW : 192.168.1.1 (ip route)

puis on va éteindre mycellius-dev et mycellius-prod en gardant mycellius-test allumé:

- mettre à jour /etc/hosts:

```
sudo vi /etc/hosts
```

- Remplace la ligne avec mycellius-dev par mycellius-test

- puis on va éteindre mycellius-dev et mycellius-test en gardant mycellius-prod allumé

- Avant de configurer l'IP static, repère le nom de l'interface : souvent ens18.

- Tu peux vérifier avec la commande :

```
ip link
```

On va maintenant chercher le fichier netplan

```
ls /etc/netplan
```

Tu verras un fichier du type :

```
50-cloud-init.yaml
```

Édite le :

```
sudo vi /etc/netplan/50-cloud-init.yaml
```

Exemple de config statique (à adapter)

- Ensuite mets (exemple) :

network:
version: 2
ethernets:
ens18:
 dhcp4: no
 addresses: [192.168.1.51/24]
 gateway4: 192.168.1.1
 nameservers:
 addresses: [1.1.1.1, 8.8.8.8]

✅ Ici on a juste changé 192.168.1.51 → 192.168.1.52

Appliquer la modification :

```
sudo netplan generate  
sudo netplan apply
```

On vérifie :

```
hostname  
hostname -I  
ip route
```

✅ Résultat attendu : mycellius-test + IP ...52

On fait la même pour mycellius-prod 192.168.1.51 → 192.168.1.53

```
clarisse@mycellius-test:~$ hostname  
mycellius-test  
clarisse@mycellius-test:~$ hostname -I  
192.168.1.52 172.17.0.1 2a01:cb08:834c:7e00:1c3c:51ff:fe35:fa6f  
clarisse@mycellius-test:~$ ip route  
default via 192.168.1.1 dev ens18 proto static  
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown  
192.168.1.0/24 dev ens18 proto kernel scope link src 192.168.1.52
```

Checkpoint (avant déploiement Git / Nginx)

```
clarisse@mycellius-prod:~$ hostname  
mycellius-prod  
clarisse@mycellius-prod:~$ hostname -I  
192.168.1.53 172.17.0.1 2a01:cb08:834c:7e00:cce2:31ff:fe62:446c  
clarisse@mycellius-prod:~$ ip route  
default via 192.168.1.1 dev ens18 proto static  
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown  
192.168.1.0/24 dev ens18 proto kernel scope link src 192.168.1.53
```

On a maintenant :

IP mycellius-dev : 192.168.1.51

IP mycellius-test : 192.168.1.52

IP mycellius-prod : 192.168.1.53

Étape 6 — Déploiement “preuve” sur chaque VM (git clone par branche → /opt/mycellius → copie → /var/www/html)(Nginx)

a. Installer Git sur chaque VM (si pas déjà fait)

Où ? SSH/console sur dev + test + prod

```
sudo apt update
```

```
sudo apt install -y git
```

Vérifier nginx actif :

```
sudo systemctl status nginx --no-pager
```

✅ attendu : active (running)

Vérifier que le firewall laisse passer HTTP (si UFW activé) :

```
sudo ufw status
```

Si tu ne vois pas Nginx autorisé :

```
sudo ufw allow 'Nginx Full'
```

```
clarisse@mycellius-dev:~$ sudo ufw status
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
Nginx Full ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
Nginx Full (v6) ALLOW Anywhere (v6)
```

b. Préparer une page HTML différente par branche

Où ? Sur ton poste (repo Git) ou directement sur GitHub

Dans ton repo, fichier : infra/static/index.html

Sur vscode, branch dev (github -> badge DEV)

Ex de contenu dans une architecture html normale (j'attends le corps d'un fichier html de base):

```
<title>Mycellius — DEV</title>
```


<p>ENV : DEV</p>

Commit + push sur dev :

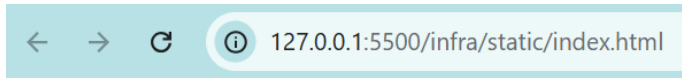
git checkout dev

git add infra/static/index.html ou git add . (si plusieurs fichiers)

git commit -m "docs: add dev static proof page"

git push

Go live pour voir



ENV : DEV

Objectif des PR (simple)

> Tu travailles et commits sur dev

> Tu “promueux” ce qui est prêt via une PR dev → test

> Puis tu “promueux” via une PR test → prod

Rappel — PR

Une Pull Request = “Je propose d’intégrer le contenu de A vers B”.

GitHub te montre les changements et tu cliques Merge.

Pré-requis : ton repo est ouvert dans VS Code

Où ? Poste Wi-Fi → VS Code

Dans le terminal VS Code (idéalement Git Bash), commence par te mettre propre :

git status

git fetch --all

git fetch --all : à quoi ça sert ?

Ça télécharge les infos des branches distantes depuis GitHub (origin), sans modifier tes fichiers.

Ça te permet de savoir si origin/dev, origin/test, origin/prod ont bougé.

Contrairement à git pull, ça ne fusionne rien : c’est “lecture seule”.

Résultat attendu :

- souvent aucune sortie (c'est normal)
- ou une liste de nouvelles infos/branches récupérées

Faire la PR dev → test (promotion)

Où ? Dans le navigateur sur GitHub (pas dans VS Code)

a. Créer la Pull Request

Va sur ton repo GitHub:

Clique l'onglet Pull requests

Clique New pull request

En haut, règle bien :

base : test  (la branche qui reçoit)

compare : dev  (la branche qui envoie)

Clique Create pull request

Rappel — base/compare

base = destination (test)

compare = source (dev)

b. Vérifier le contenu

Sur l'écran de la PR, tu dois voir que le fichier modifié inclut infra/static/index.html avec "DEV".

c. Merger

Clique Merge pull request puis Confirm merge

Mettre la page "TEST" sur la branche test (via une mini-PR propre)

Où ? VS Code (terminal + éditeur) puis GitHub (navigateur)

a. Se placer sur test et la mettre à jour

Dans le terminal VS Code :

```
git checkout test
```

```
git pull origin test
```

À quoi ça sert ?

checkout test : tu passes sur la branche test localement

pull origin test : tu récupères la version actuelle de test (après le merge)



Check rapide :

```
git status
```

(attendu : "On branch test")

b. Créer une petite branche de travail "badge-test"

```
git checkout -b badge-test
```

À quoi ça sert ?

Tu crées une branche dédiée à une modification spécifique à l'environnement test, sans polluer l'historique de dev.

c. Modifier le fichier pour afficher TEST

Dans VS Code, ouvre infra/static/index.html et remplace :

Mycellius — DEV → Mycellius — TEST

ENV : DEV → ENV : TEST

branche dev → branche test (optionnel mais mieux)

d. Commit + push de badge-test

```
git add infra/static/index.html
```

```
git commit -m "docs: env badge TEST"
```

```
git push -u origin badge-test
```

À quoi ça sert ?

add/commit : tu enregistres la modif

push -u : tu publies la branche sur GitHub et tu la "lies" à la branche distante

e. PR badge-test → test (sur GitHub)

Où ? navigateur GitHub

Pull requests → New pull request

Choisis :

base : test

compare : badge-test

Create PR → Merge

Promouvoir test → prod (PR de promotion)

Où ? GitHub (navigateur)

Va dans Pull requests

New pull request

En haut, choisis :

base : prod

compare : test

Create pull request

Merge pull request → Confirm

À quoi ça sert ?

C'est la "promotion" officielle : ce qui est validé en test devient candidat prod.

Mettre la page "PROD" sur la branche prod (via mini-PR)

Où ? VS Code (terminal + éditeur) puis GitHub

a. Passer sur prod et récupérer la dernière version

Dans le terminal VS Code :

```
git checkout prod
```

```
git pull origin prod
```

```
git status
```

À quoi ça sert ?

- se placer sur prod

- récupérer la version à jour (après la PR test→prod)

- vérifier que tout est clean

✅ Attendu : "On branch prod" et working tree clean.

b. Créer une branche dédiée "badge-prod"

```
git checkout -b badge-prod
```

À quoi ça sert ?

Créer une branche de travail spécifique à la config "prod".

c. Modifier infra/static/index.html pour afficher PROD

Remplace :

Mycellius — TEST ou DEV → Mycellius — PROD

ENV : ... → ENV : PROD

et la phrase "branche" → prod

d. Commit + push

```
git add infra/static/index.html
```

```
git commit -m "docs: env badge PROD"
```

```
git push -u origin badge-prod
```

e.) PR badge-prod → prod (sur GitHub)

Pull requests → New pull request

base : prod

compare : badge-prod

Create → Merge

Pour résumé, les procédures sont les suivantes :

- Promouvoir vers test puis adapter TEST

Fais une PR dev → test, merge.

Sur test, change le badge en TEST, commit, push sur test :

```
git checkout test
```

```
# modifier infra/static/index.html -> ENV TEST
```

```
git add infra/static/index.html  
git commit -m "docs: env badge TEST"  
git push
```

Promouvoir vers prod puis adapter PROD

PR test → prod, merge.

Sur prod, change le badge en PROD, commit, push sur prod :

```
git checkout prod  
# modifier infra/static/index.html -> ENV PROD  
git add infra/static/index.html  
git commit -m "docs: env badge PROD"  
git push
```

✅ **À la fin : chaque branche a sa page "ENV".**

Déployer DEV/TEST/PROD sur les 3 VM (Nginx sert /var/www/html)

Avant tout : le repo sur GitHub doit contenir les 3 pages

branche dev → page affiche DEV

branche test → page affiche TEST

branche prod → page affiche PROD

a. Commandes à exécuter sur CHAQUE VM (en SSH)

Tu vas te connecter en SSH depuis ton poste Wi-Fi.

VM DEV (192.168.1.51)

Où ? Terminal sur ton poste :

```
ssh <user>@192.168.1.51
```

Puis dans la VM :

```
sudo rm -rf /opt/mycellius  
sudo git clone -b dev <URL_DU_REPO> /opt/mycellius
```

```
sudo rm -rf /var/www/html/*  
  
sudo cp -r /opt/mycellius/infra/static/* /var/www/html/  
  
sudo systemctl reload nginx
```

VM TEST (192.168.1.52)

Connexion :

```
ssh <user>@192.168.1.52
```

Puis :

```
sudo rm -rf /opt/mycellius  
  
sudo git clone -b test <URL_DU_REPO> /opt/mycellius  
  
sudo rm -rf /var/www/html/*  
  
sudo cp -r /opt/mycellius/infra/static/* /var/www/html/  
  
sudo systemctl reload nginx
```

VM PROD (192.168.1.53)

Connexion :

```
ssh <user>@192.168.1.53
```

Puis :

```
sudo rm -rf /opt/mycellius  
  
sudo git clone -b prod <URL_DU_REPO> /opt/mycellius  
  
sudo rm -rf /var/www/html/*  
  
sudo cp -r /opt/mycellius/infra/static/* /var/www/html/  
  
sudo systemctl reload nginx
```

À quoi ça sert ?

git clone -b X ... /opt/mycellius : récupère la branche X sur cette VM (donc bon environnement)

cp ... /var/www/html/ : met uniquement les fichiers statiques dans le dossier servi par nginx

reload nginx : recharge nginx pour prendre en compte les fichiers (souvent pas obligatoire, mais propre)

b. Test navigateur (poste Wi-Fi)

Ouvre :

<http://192.168.1.51> → doit afficher DEV

<http://192.168.1.52> → doit afficher TEST

<http://192.168.1.53> → doit afficher PROD

Étape 8 - CI minimale (GitHub Actions)

Pourquoi on fait ça ? Faisons une analogie...

Ton dépôt GitHub devient une maison en construction.

Avant d'ajouter l'électricité (API), la plomberie (DB), et la déco (front), tu veux :

- des plans rangés (docs/)
- un chantier organisé (infra/)
- des règles de circulation (branches dev/test/prod + PR)
- un contrôle qualité à l'entrée (CI)
- Cette étape met le contrôle qualité automatique + la base cybersécurité.

Étape CI-1 — Dossier ci/ et script de validation (local, VS Code)

Où ? Poste Wi-Fi → VS Code (dans ton repo)

1) Vérifie que tu es sur dev et propre

Dans le terminal VS Code :

```
git checkout dev
git status
```

✓ Attendu : On branch dev + working tree clean (ou au moins pas de surprise).

2) Dossier ci/ et le script validate_repo.sh

Liste le contenu du répertoire mycellius, tu devrais avoir le répertoire **ci** dans la liste car nous l'avons créé à l'étape **1D) Créer la structure en local + push sur dev en page 13**.

```
clakg@CKG-FORMATION MINGW64 ~/mycellius (dev)
$ ls -l
total 0
drwxr-xr-x 1 clakg 197609 0 Dec 2 23:47 api/
drwxr-xr-x 1 clakg 197609 0 Dec 2 23:47 ci/
drwxr-xr-x 1 clakg 197609 0 Dec 2 23:47 docs/
drwxr-xr-x 1 clakg 197609 0 Dec 2 23:47 infra/
drwxr-xr-x 1 clakg 197609 0 Dec 2 23:47 mobile/
-rw-r--r-- 1 clakg 197609 0 Dec 3 23:58 README.md
drwxr-xr-x 1 clakg 197609 0 Dec 2 23:47 web/
```


Puis crée **un fichier validate_repo.sh** dans le répertoire **ci** (dans VS Code) avec ce contenu :

```
#!/usr/bin/env bash
set -euo pipefail
echo "== Vérification structure du dépôt =="
test -f README.md
test -d docs
test -d infra
test -d .github/workflows
test -f docs/01_cadrage.md
test -f docs/02_architecture.md || true
test -f docs/03_securite_baseline.md || true
test -d infra/static
test -f infra/static/index.html
echo "OK: structure minimale présente."
```

Rappel — à quoi sert ce script ?

test -f / test -d : échoue si un fichier/dossier manque

set -euo pipefail : rend le script “strict” (arrêt au premier problème)

En gros :

Le script validate_repo.sh est un contrôle technique ultra simple : il ne teste pas ton application (elle n'existe pas encore), mais il vérifie que ton dépôt contient bien ce qu'il doit contenir.

test -f README.md = “si le fichier n'existe pas → échec”

test -d docs = “si le dossier n'existe pas → échec”

Donc si quelqu'un supprime docs/ ou oublie README.md, la CI se met en rouge et te le signale tout de suite.

set -euo pipefail

Ça rend le script strict :

-e : stop dès qu'une commande échoue

-u : stop si une variable est utilisée alors qu'elle n'existe pas

-o = “option” : active l'option pipefail

pipefail : si une commande dans un “pipe” échoue, le script le considère comme un échec

=> ça empêche les “faux verts”. On veut que ce soit fiable. Donc la CI peut dire “vert/rouge” très tôt, même sans application.

3) Rendre le script exécutable (*s'il ne l'est pas déjà...*)

Avant d'appliquer un droit d'exécution à ce fichier, vérifier les droits.

Mais comment fait-on pour vérifier les droits ? Je vous laisse chercher un peu...

Si vous avez trouvé et les droits d'exécution ne sont pas appliqués, alors faites le :

```
chmod +x ci/validate_repo.sh
```

CI-2 — Workflow GitHub Actions ci.yml

Rappel — c'est quoi un "workflow" GitHub Actions ?

Un workflow, c'est une recette automatique que GitHub exécute sur ses serveurs dès qu'un événement arrive (push, PR...).

Dans notre TP, c'est ton contrôle qualité automatique : il vérifie que le dépôt garde une structure correcte (docs, infra, etc.), même si l'appli n'est pas encore codée.

Ce fichier, on l'a déjà en même temps que tous les autres. c'est la notice d'automatisation pour GitHub.

```
mkdir -p .github/workflows
```

- GitHub Actions ne lit que les workflows placés dans ce dossier.

- C'est comme dire : "voici l'endroit officiel où GitHub va chercher les règles CI".

En gros, tu automatises le prof/chef de projet qui dit :

"Avant d'accepter ton travail dans test ou prod, je vérifie qu'il ne manque pas les fichiers de base."

Où ? Poste Wi-Fi → VS Code (repo) → fichier à créer

1) Créer le dossier workflow (si pas encore créé mais normalement si...)

Dans le terminal VS Code :

```
mkdir -p .github/workflows
```

À quoi ça sert ?

GitHub Actions détecte automatiquement les workflows placés dans `.github/workflows/`.

2) Créer le fichier `.github/workflows/ci.yml`

Crée (ou ouvre) ce fichier dans VS Code et colle :

```

name: CI

on:
  push:
    branches: [dev, test, prod]
  pull_request:
    branches: [test, prod]

jobs:
  validate-repo:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Validate repository structure
        run: bash ci/validate_repo.sh

```

Le YAML ci.yml décrit :

Rappel — à quoi servent ces blocs et ces lignes ?

- name: CI

Juste le nom affiché dans l'onglet Actions.

- on:

C'est le "déclencheur".

> push sur dev/test/prod : dès que tu pushes, GitHub lance la CI.

> pull_request vers test/prod : quand tu proposes une PR vers test ou prod, GitHub vérifie avant merge.

- jobs:

Un workflow contient des jobs.

Un job = un "petit ordinateur" sur GitHub qui exécute des étapes.

> runs-on: ubuntu-latest

GitHub lance ton job sur une VM Ubuntu (standard).

> actions/checkout@v4

Étape indispensable : GitHub télécharge ton dépôt dans la VM du job.

Sans ça, il n'aurait pas tes fichiers, donc validate_repo.sh ne peut pas s'exécuter.

> run: bash ci/validate_repo.sh

GitHub exécute ton script.

Si un fichier attendu manque → le script échoue → la CI passe en rouge.

3) Vérifier que le fichier existe

```
ls -la .github/workflows
```

CI-3 — Commit + push du workflow sur dev (puis vérification Actions)

Où ? Poste Wi-Fi → VS Code → terminal

1) Vérifier ce qui va être committé

```
git status
```

À quoi ça sert ?

Voir les fichiers modifiés/non suivis (on évite d'envoyer un truc inattendu).

2) Ajouter les fichiers de CI

```
git add ci/validate_repo.sh .github/workflows/ci.yml
```

À quoi ça sert ?

Mettre ces fichiers dans le "staging" pour le commit.

3) Commit

```
git commit -m "chore(ci): add minimal GitHub Actions workflow"
```

À quoi ça sert ?

Enregistrer un point d'historique clair : "on a ajouté la CI".

4) Push sur GitHub (branche dev)

```
git push origin dev
```

À quoi ça sert ?

Déclencher GitHub Actions (car le workflow est poussé).

Vérifier sur GitHub

Mini mémo des types courants

feat: → nouvelle fonctionnalité utilisateur

fix: → correction de bug

docs: → documentation

chore: → maintenance/outillage (CI, configs, scripts...)

test: → ajout/modif de tests

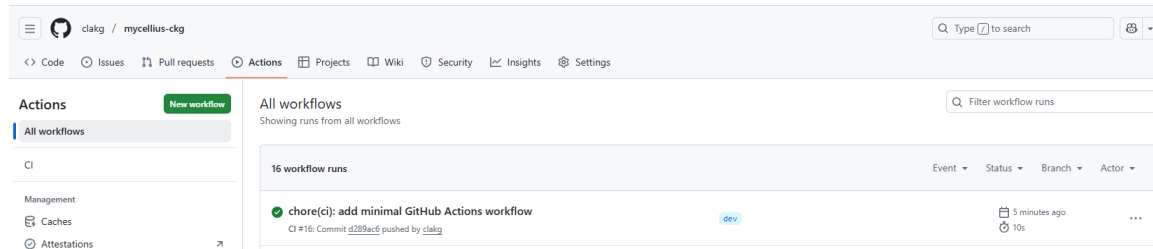
refactor: → restructuration du code sans changer le comportement

Où ? Navigateur → repo GitHub → onglet Actions

Tu dois voir un run “CI” :

✅ vert = OK

❌ rouge = un fichier manque ou script non exécutable



Concrètement, après l'étape 8, ton projet passe de “on travaille à la main” à “on travaille avec des garde-fous”, exactement comme en entreprise.

1) À chaque push sur dev/test/prod, GitHub lance automatiquement un contrôle

Dès que tu fais git push :

- GitHub démarre un run dans l'onglet Actions
- il récupère ton dépôt (checkout)
- il exécute ton script ci/validate_repo.sh

✅ Si tout est conforme → vert

❌ S'il manque un fichier/dossier attendu → rouge (et tu sais que le dépôt n'est plus “au standard”)

Exemple réel : un étudiant supprime par erreur infra/static/index.html → la CI passe en rouge → tu le vois tout de suite.

2) À chaque Pull Request vers test/prod, la CI se lance avant le merge

Quand tu ouvres une PR :

- GitHub vérifie automatiquement que le dépôt respecte la structure attendue
- si c'est rouge, tu évites de “promouvoir” quelque chose de bancal vers test/prod
- C'est le garde-barrière avant d'entrer en “test” puis en “prod”.

3) Tu as un socle prêt pour la suite (API / web / mobile)

Dès que vous commencerez à coder :

- tu remplaceras/complèteras le script de validation par de vrais jobs : build Spring, tests JUnit, build React, etc.

mais la mécanique est déjà en place : push / PR → CI → vert/rouge

4) Résultat final de Sprint 0

À la fin :

- dépôt structuré
- workflow dev→test→prod opérationnel
- VM dev/test/prod qui affichent leur environnement
- CI en place (même “simple”)
- baseline sécurité écrite (ce qu'on va faire maintenant)

Etape 9 - baseline sécurité (CIA + mini registre des risques)

Le doc CIA + registre des risques te donne :

- une démarche cybersécurité claire (Confidentialité / Intégrité / Disponibilité)
- une liste de risques + mesures + preuves

Plus tard, tu complèteras avec JWT/RBAC, logs, audits, backups... mais la base est déjà posée et ajustable.

Sécurité-1 — Créer docs/03_securite_baseline.md (CIA + risques)

Où ? Poste Wi-Fi → VS Code (branche dev)

1) Créer le fichier (déjà créé en 1D mais si)

Crée : docs/03_securite_baseline.md

2) Colle ce gabarit (tu pourras l'adapter ensuite)

Séance 1 — Baseline cybersécurité (Bloc 3) : CIA + registre des risques

1) CIA (Confidentialité, Intégrité, Disponibilité)

Confidentialité (C)

Objectif : empêcher l'accès aux données par des personnes non autorisées.

Mesures minimales (Sprint 0 / à venir) :

- Authentification (JWT) et rôles (RBAC) à partir de la séance sécurité.
- Secrets hors dépôt Git (pas de mots de passe dans le code, .env non commité).
- Accès serveur via SSH par clé (pas de mot de passe faible).
- HTTPS en production (plus tard).

Preuves :

- Clés SSH en place sur les VM.
- `.gitignore` + `.env.example` (si utilisé) et aucun secret commité.

Intégrité (I)

Objectif : empêcher la modification non autorisée et garantir un état cohérent.

Mesures minimales :

- Validation côté serveur (à venir avec Spring).
- Migrations de base de données versionnées (Flyway/Liquibase, à venir).

- CI qui vérifie la structure et qui échouera si la base manque.
- Historique Git + PR dev→test→prod.

Preuves :

- Workflow Git (PR) actif.
- GitHub Actions "CI" en vert.

Disponibilité (A)

Objectif : garder le service accessible et récupérable après incident.

Mesures minimales :

- 3 environnements séparés (dev/test/prod) : une casse n'impacte pas prod.
- Procédure de redémarrage services (nginx, docker).
- Sauvegarde/restauration (à venir avec MySQL).

Preuves :

- 3 VM accessibles et pages DEV/TEST/PROD.
- Commandes de restart connues/documentées.

2) Registre des risques (mini)

Risque	Impact	Vraisemblance	Mesures de réduction	Preuve / contrôle
Secrets committés (token, mdp)	Fort	Moyen	`.env` non versionné, variables d'environnement, relecture PR	`.gitignore`, revue PR, éventuel secret scanning
Compromission SSH (mot de passe faible)	Fort	Moyen	Auth par clé, désactiver password login (option)	test connexion par clé, config ssh
Perte de données DB	Fort	Moyen	backups + test de restauration	procédure + preuve test
Injection / XSS via entrées utilisateur	Fort	Moyen	validation serveur + encodage + tests sécurité	tests, ZAP plus tard
Token JWT volé	Fort	Moyen	durée courte, rotation, stockage sécurisé (mobile)	config JWT + SecureStore

Secrets committés (token, mdp) | Fort | Moyen | `.env` non versionné, variables d'environnement, relecture PR | `.gitignore`, revue PR, éventuel secret scanning |

Compromission SSH (mot de passe faible) | Fort | Moyen | Auth par clé, désactiver password login (option) | test connexion par clé, config ssh |

Perte de données DB | Fort | Moyen | backups + test de restauration | procédure + preuve test |

Injection / XSS via entrées utilisateur | Fort | Moyen | validation serveur + encodage + tests sécurité | tests, ZAP plus tard |

Token JWT volé | Fort | Moyen | durée courte, rotation, stockage sécurisé (mobile) | config JWT + SecureStore |

3) Décisions prises en séance 1 (Sprint 0)

- Branches : dev/test/prod avec PR obligatoires vers test/prod.
- 3 VM : 192.168.1.51 (DEV), 192.168.1.52 (TEST), 192.168.1.53 (PROD).
- “Preuve de déploiement” : nginx sert une page différente selon la branche.

Rappel — pourquoi on écrit ça maintenant ?

Parce que le Bloc 3 demande une démarche cybersécurité : on prouve qu’on a réfléchi dès le début (et pas “à la fin quand on a le temps”).

Sécurité-2 — Commit + push sur dev, puis PR → test, puis PR → prod

Où ? Poste Wi-Fi → VS Code (terminal) + GitHub (navigateur)

1) Commit + push sur dev (VS Code)

Dans le terminal :

1A) Vérifier l’état

```
git status
```

But : vérifier que docs/03_securite_baseline.md est bien détecté.

1B) Ajouter le fichier

```
git add docs/03_securite_baseline.md
```

But : sélectionner ce fichier pour le commit.

1C) Commit

```
git commit -m "docs: add security baseline (CIA + risks)"
```

But : enregistrer la baseline sécurité dans l’historique.

1D) Push

```
git push origin dev
```

But : envoyer sur GitHub et déclencher la CI.

 Vérifie vite dans GitHub Actions : le run “CI” doit être vert.

2) Créer la Pull Request : PR dev → test (GitHub)

Où ? GitHub → onglet Pull requests

Rappel :

base = la branche qui reçoit (destination)

compare = la branche qui envoie (source)

- New pull request

base = test ; compare = dev

- Clique sur Create pull request

- Titre : Promote dev to test (security baseline)

- Clique ensuite sur Create pull request (oui, il y a parfois 2 boutons "Create" selon l'écran)

- Merger la PR : Sur la page de la PR, tu dois voir le bouton :

Clique dessus → Merge pull request

Clique dessus → puis Confirm merge

=> But : promouvoir la doc sécurité vers l'environnement de test.

✅ Après merge : CI doit passer sur test aussi.

Résultat attendu :

- la PR passe en "Merged"

- la branche test contient maintenant la doc sécurité

Vérifier la CI sur test

- Clique l'onglet Actions (en haut du repo)

- Regarde le dernier run "CI"

- vert = OK

- rouge = on corrige

3) PR test → prod (GitHub)Toujours Pull requests :

- New pull request

- base = prod ; compare = test

=> donc : test est promu vers prod

- Créer la PR : Clique Create pull request

- titre : Promote test to prod (security baseline)

- puis Clique le bouton "Create pull request"

- puis Merge pull request puis Confirm merge

=> But : promouvoir vers prod (même les docs doivent suivre le cycle).